

FYP Pre-Start Preparation Plan (Revised)

LLM-Based Interface for Robotics Systems using UR5E | Period: 16 June 2026 to 4 August 2026 (6.5 weeks) | Target: 15 hours/week

What changed in this revision

This version aligns the plan with decisions made with the supervisor and during scoping. The project now centres on a comparison between a modular LLM-driven pipeline and an end-to-end Vision-Language-Action (VLA) model, with generalisation to unseen objects as the evaluation focus. Three changes follow from this: a perception and modular-pipeline thread has been added (open-vocabulary detection, depth-to-3D localisation, hand-eye calibration, scripted motion primitives); the egocentric-video hand-pose hands-on work has been removed, since demonstrations will be collected directly on the UR5E (EgoMimic and UMI remain as background reading only); and a closed-loop control strategy is introduced. Weeks 1 to 5 are largely unchanged, as their foundations remain correct.

Weekly time budget

Slot	Duration	Notes
Mon-Fri (each)	1.5 hours	After work, focused single task per session
Saturday	1-5 hours	Variable - reading or moderate hands-on
Sunday	6-8 hours	Full day, biggest building block of the week
Weekly total	~15 hours	Adjust up or down based on actual capacity

Six-week arc at a glance

Week	Dates	Focus
1	16-22 Jun	UR fundamentals + Polyscope basics (e-Series Core Track)
2	23-29 Jun	URScript language + kinematics (FK, IK, rotations, frames)
3	30 Jun - 6 Jul	Python to UR via ur_rtde; build the URController class
4	7-13 Jul	VLA paper reading (RT-2, OpenVLA, OXE) + ManiSkill3 sim
5	14-20 Jul	SmolVLA + LeRobot reproduction + RTC paper
6	21-27 Jul	Perception + modular pipeline + LLM planner (closed-loop slice)
6.5	28 Jul - 4 Aug	Proposal refinement, consolidation, supervisor sign-off

Note: the proposal has already been drafted ahead of schedule, so Week 6.5 shifts from writing to refinement and consolidation.

Week 1 — UR fundamentals and Polyscope basics

Week	1	Dates	16 - 22 June 2026
Target hours	~15 hours		
Theme	Mental model of UR robot + Polyscope UI fluency		

Goal: by Sunday night you can navigate Polyscope confidently, explain what TCP and payload mean, and have built and run a basic pick-and-place program in URSim. Use the e-Series Core Track on UR Academy (NOT PolyScope X, which is for UR20/UR30 only).

Monday 16 Jun (1.5 hrs) — UR Academy account + Module 1

- [] Create account at academy.universal-robots.com
- [] Locate the e-Series Core Track (search 'e-Series' or 'UR5e')
- [] Complete Module 1: Hardware Overview
- [] Write notes: name all 6 joints (base, shoulder, elbow, wrist 1/2/3)
- [] Note key specs: payload 5 kg, reach 850 mm, 6 DOF

Note: If you cannot find e-Series Core Track, search for 'Polyscope 5' training instead.

Tuesday 17 Jun (1.5 hrs) — Module 2: Robot setup + TCP concept

- [] Complete e-Series Module 2: Robot Setup
- [] Open URSim. Click into the Installation tab
- [] Locate and read about: TCP, Payload, Mounting, Features
- [] Write a short note explaining: what is the TCP, why does payload matter
- [] Try defining a custom TCP offset in URSim Installation tab

Wednesday 18 Jun (1.5 hrs) — Robotic motion concepts

- [] Read Modern Robotics (Lynch) Ch 2.1-2.2 - configuration space and DOF
- [] Watch a short intro video on UR robot joints if Lynch is too dense
- [] Sketch a diagram of UR5e with the 6 joints labeled
- [] Write 3-5 sentences explaining what 'degrees of freedom' means physically

Thursday 19 Jun (1.5 hrs) — Module 3: Movement types

- [] Complete e-Series Module 3: Movement and Motion Types
- [] Note differences: MoveJ (joint space, curved), MoveL (linear in tool space), MoveP (constant velocity)
- [] In URSim: create a program with one MoveJ and one MoveL between the same two waypoints
- [] Run both and observe the path difference

Friday 20 Jun (1.5 hrs) — First Polyscope program

- [] Complete e-Series Module 4: Creating a Program
- [] In URSim: build a 3-waypoint program (home, pick, place)
- [] Add a Wait command and a Set command (gripper simulation)
- [] Save the .urp file and run it

Saturday 21 Jun (2 hrs) — Rotation representations

- [] Read Modern Robotics Ch 3.2 (Rotations and Angular Velocities)
- [] Focus specifically on axis-angle representation
- [] Understand: $p[x,y,z,r_x,r_y,r_z]$ where $[r_x,r_y,r_z]$ is a rotation vector
- [] Note: magnitude of $[r_x,r_y,r_z]$ is rotation angle in radians, direction is the axis

Note: Do NOT try to master rotation math this week. Just understand the URScript pose format.

Sunday 22 Jun (6 hrs) — Full Polyscope build + consolidation

- [] Complete remaining e-Series Core modules
- [] In URSim: build a program with variables and an If statement
- [] Try a Subprogram and a Loop construct
- [] Save all programs to a dedicated folder
- [] Write a 1-page summary: TCP, payload, MoveJ vs MoveL, axis-angle, .urp vs .installation

Week 1 — deliverable checklist and notes

- ☐ UR Academy e-Series Core Track marked complete
- ☐ At least 2 working .urp programs saved
- ☐ 1-page summary written
- ☐ Can answer on the spot: 'what is the TCP?' and 'MoveJ vs MoveL?'

What I want to ask my supervisor

My notes

Week 2 — URScript language and kinematics concepts

Week	2	Dates	23 - 29 June 2026
Target hours	~15 hours		
Theme	Write URScript programs, understand FK/IK		

Goal: by Sunday night you can write URScript programs from scratch in the Script tab, you understand the difference between forward and inverse kinematics, and you can explain what a singularity is. Download the URScript Programming Manual for e-Series from Universal Robots' website.

Monday 23 Jun (1.5 hrs) — URScript manual - introduction

- [] Download 'URScript Programming Manual e-Series' PDF
- [] Read Sections 1-2 (intro, syntax basics)
- [] Note key functions: movej, movel, pose_trans, get_actual_tcp_pose, get_actual_joint_positions
- [] In URSim: write a 3-line hello-world in the Script tab (sleep, print, sleep)

Tuesday 24 Jun (1.5 hrs) — URScript motion commands

- [] URScript Manual Section 4 - motion commands
- [] Understand movej(q, a, v) - q is joint vector (6 floats in radians)
- [] Understand movel(p, a, v) - p is pose [x,y,z,rx,ry,rz]
- [] Write URScript: move to an absolute joint config, then a relative TCP offset

Wednesday 25 Jun (1.5 hrs) — Forward kinematics

- [] Read Modern Robotics Ch 4.1 - forward kinematics intro
- [] Concept: given joint angles q, FK returns TCP pose p
- [] In URSim: print get_actual_joint_positions() and get_actual_tcp_pose() together
- [] Move the robot in freedrive and observe both values change
- [] Write 1 paragraph explaining FK in your own words

Thursday 26 Jun (1.5 hrs) — Inverse kinematics + singularities

- [] Read Modern Robotics Ch 6.1 - inverse kinematics intro
- [] Concept: given a target TCP pose, IK finds joint angles to achieve it
- [] Understand: multiple solutions possible (elbow up/down, wrist flips)
- [] Read about wrist, shoulder, and elbow singularities
- [] Write 1 paragraph on why IK can fail or have multiple answers

Friday 27 Jun (1.5 hrs) — URScript control flow

- [] URScript Manual Section 5 - variables, if/else, while loops
- [] Write a URScript for-loop visiting 4 corners of a square in TCP space
- [] Use pose_trans to compute corners relative to a starting pose

Saturday 28 Jun (2 hrs) — Coordinate frames deep dive

- [] Study frames: base, TCP, world, tool, feature frames
- [] Understand: a pose is always relative to SOME frame - which one?
- [] Read about pose_trans(p_from, p_offset) and pose_inv(p)
- [] Draw a diagram relating base frame and TCP frame via the joints

Note: This frame fluency feeds directly into hand-eye calibration in Week 6.

Sunday 29 Jun (6 hrs) — Big URScript exercise + RT-1 skim

- [] Write a comprehensive URScript pick-and-place in URSim that:
- [] ... starts from a known home pose
- [] ... picks an object at a defined location (simulated gripper)

- [] ... moves through a safe waypoint
- [] ... places at a destination defined as an offset from a feature
- [] ... returns home
- [] Use pose_trans for all offsets (not hardcoded poses)
- [] Light reading: skim RT-1 (Brohan et al. 2022), 30 min, gist only
- [] Take 5-10 bullets on RT-1: input, output, architecture, key result

Week 2 — deliverable checklist and notes

- ☐ URScript pick-and-place program working in URSim
- ☐ Notes on FK, IK, singularities (1 page)
- ☐ RT-1 skim notes
- ☐ Can explain: 'why might IK return no solution for a reachable-looking pose?'

What I want to ask my supervisor

My notes

Week 3 — Python to UR communication

Week	3	Dates	30 June - 6 July 2026
Target hours	~15 hours		
Theme	Build the URController Python class		

Goal: by Sunday night you have a clean Python class (URController) wrapping ur_rtde with high-level methods (move_to_pose, move_joints, get_state, set_gripper). This is the foundation your entire FYP code sits on - make it clean.

Monday 30 Jun (1.5 hrs) — Environment setup + ur_rtde install

- [] Create a Python 3.10/3.11 virtual environment for the FYP
- [] Install ur_rtde: `pip install ur_rtde`
- [] Initialise a git repository
- [] Set up folders: `src/`, `notebooks/`, `tests/`, `data/`, `docs/`
- [] Write a README.md stub

Tuesday 1 Jul (1.5 hrs) — ur_rtde basics

- [] Read ur_rtde docs: sdurbotics.gitlab.io/ur_rtde
- [] Connect to URSim from Python (localhost or container IP)
- [] Read TCP pose via `getActualTCPPOSE()` and joints via `getActualQ()`
- [] Print both; verify they match URSim

Wednesday 2 Jul (1.5 hrs) — Sending motion from Python

- [] Use `moveL()` to move the TCP to a target pose
- [] Use `moveJ()` to move to a target joint config
- [] Script motion through 3 predefined poses in sequence
- [] Add error handling for connection failures

Thursday 3 Jul (1.5 hrs) — RTDE concepts

- [] Read about the Real-Time Data Exchange protocol
- [] Understand the 500 Hz state stream
- [] Configure a custom RTDE recipe (which signals to receive)
- [] Log joint positions to CSV at 100 Hz for 5 seconds

Friday 4 Jul (1.5 hrs) — Dashboard server

- [] Read about the Dashboard server (port 29999)
- [] Use Python sockets to send: power on, brake release, play
- [] Write a Python wrapper for dashboard commands
- [] Test: load a saved .urp program and play it from Python

Saturday 5 Jul (3 hrs) — RT-2 paper - first deep read

- [] Read RT-2 (Brohan et al. 2023) - intro and method
- [] Notes: input space (image + language), output (discretised action tokens)
- [] Note the key innovation: a pretrained VLM as backbone
- [] Read experiments and results; note the headline generalisation claim

Note: RT-2's unseen-object generalisation is the exact claim your FYP will test.

Sunday 6 Jul (6 hrs) — Build the URController class

- [] Design URController with methods:
- [] ... `__init__(host, port)` - connect via ur_rtde
- [] ... `move_to_pose(pose, speed, accel)` - linear move

- [] ... move_joints(q, speed, accel) - joint move
- [] ... get_state() - dict with joint_pos, tcp_pose, gripper_state
- [] ... set_gripper(open: bool) - placeholder/log for now
- [] ... close() - clean disconnect
- [] Add docstrings and type hints
- [] Write a test script exercising every method
- [] Commit to git; aim for 200-300 lines of clean code

Week 3 — deliverable checklist and notes

- ☐ URController class works with URSim
- ☐ Git repo initialised, code committed
- ☐ RT-2 paper read and notes taken
- ☐ Can demonstrate the controller from a Jupyter notebook

What I want to ask my supervisor

My notes

Week 4 — Deep VLA reading and simulation

Week	4	Dates	7 - 13 July 2026
Target hours	~15 hours		
Theme	OpenVLA + Open X-Embodiment + ManiSkill3		

Goal: by Sunday night you can explain OpenVLA's architecture on a whiteboard, you have ManiSkill3 running with a UR5e and a camera, and you have a DemoRecorder that captures synchronised (state, image) data. ManiSkill3 - not Isaac Sim - is the chosen simulator: it is lighter, supports UR5e, and is sufficient for rendering camera scenes. Isaac Sim is not required for this project.

Monday 7 Jul (1.5 hrs) — RT-2 review + transition to OpenVLA

- [] Re-read RT-2 notes: what is the action space exactly?
- [] Map RT-2's action space onto what your URController accepts
- [] Write: 'if RT-2 ran on UR5e, what would each output dimension mean?'
- [] Read OpenVLA introduction and related work

Tuesday 8 Jul (1.5 hrs) — OpenVLA - architecture

- [] Read OpenVLA (Kim et al. 2024) Sections 3-4
- [] Note: Llama 2 backbone, DINOv2 + SigLIP vision encoders
- [] Note: action representation and chunking
- [] Compare architecturally to RT-2

Wednesday 9 Jul (1.5 hrs) — OpenVLA - training and eval

- [] Read OpenVLA Sections 5-6
- [] Note training data (Open X-Embodiment subset)
- [] Note the LoRA fine-tuning recipe (you'll mirror this with SmoVLA)
- [] Read the limitations section carefully

Thursday 10 Jul (1.5 hrs) — Open X-Embodiment

- [] Read the OXE paper / data card (Padalkar et al. 2023)
- [] Focus on the action/observation data format conventions
- [] Check whether UR5 data is included and what setup it uses

Friday 11 Jul (1.5 hrs) — ManiSkill3 setup

- [] Install ManiSkill3 in a fresh conda env
- [] Verify UR5e is in the asset library
- [] Run a sample task; verify rendering works on your GPU
- [] Troubleshoot CUDA/PyTorch versions if needed

Saturday 12 Jul (3 hrs) — ManiSkill3 camera (fixed view)

- [] Spawn a UR5e scene in ManiSkill3
- [] Add a fixed third-person camera (matches your real setup) and capture RGB
- [] Read about RGB-D / depth output
- [] Save a few frames as PNG to verify

Note: Your real rig uses a FIXED camera, so prioritise the third-person view over wrist-mounted.

Sunday 13 Jul (6 hrs) — Demonstration recorder

- [] Build a DemoRecorder class capturing:
- [] ... joint_positions, tcp_pose, gripper_state, camera_image
- [] ... timestamped at a fixed rate (e.g. 20 Hz)
- [] Implement save_episode() to a clean format (HDF5 or numpy)

- [] Implement `load_episode()` for replay/inspection
- [] Test by scripting a pick motion and recording it
- [] Visualise the trajectory in matplotlib; verify images are viewable

Week 4 — deliverable checklist and notes

- ☐ OpenVLA architecture understood (can explain to a friend)
- ☐ ManiSkill3 working with UR5e and a fixed camera
- ☐ DemoRecorder class working; at least 1 episode saved and replayed

What I want to ask my supervisor

My notes

Week 5 — SmolVLA and LeRobot

Week	5	Dates	14 - 20 July 2026
Target hours	~15 hours		
Theme	Reproduce SmolVLA fine-tuning		

Goal: by Sunday night you have fine-tuned SmolVLA on the LeRobot tutorial dataset (using cloud GPU, since training exceeds an 8GB laptop), you understand each line of the training script, and you have read RTC, which addresses the control-loop latency concern. Fine-tuning is done on cloud GPU by design; the laptop handles inference and development.

Monday 14 Jul (1.5 hrs) — LeRobot installation

- [] Clone github.com/huggingface/lerobot
- [] Follow the README installation
- [] Install smolvla extras: `pip install -e '[smolvla]'`
- [] Log into HuggingFace Hub (free account)
- [] Explore the lerobot directory structure

Tuesday 15 Jul (1.5 hrs) — SmolVLA paper - first half

- [] Read SmolVLA (Shukor et al. 2025) Sections 1-3
- [] Note: ~450M params (vs 7B OpenVLA) - why it fits consumer GPUs for inference
- [] Architecture: SmolVLM-2 backbone + Action Expert
- [] Read about the asynchronous inference stack

Wednesday 16 Jul (1.5 hrs) — SmolVLA paper - second half

- [] Read SmolVLA Sections 4-6 (training, evaluation, conclusion)
- [] Note the training data source and the key ablations
- [] Write a 1-page summary of SmolVLA in your own words

Thursday 17 Jul (1.5 hrs) — LeRobot SmolVLA tutorial

- [] Open the SmolVLA fine-tuning tutorial in LeRobot docs
- [] Download the tutorial dataset
- [] Write a script printing dataset shape and episode count
- [] Visualise a few dataset frames

Friday 18 Jul (1.5 hrs) — Cloud GPU training setup

- [] Sign up for Modal Labs or Google Colab Pro
- [] Set up a training script that runs on the cloud GPU
- [] Confirm the dataset is accessible from the cloud environment
- [] Do a 1-step dry run to confirm the pipeline executes

Note: 8GB local fine-tuning is optional to try, but cloud is the planned path - don't lose time fighting OOM.

Saturday 19 Jul (3 hrs) — Run the fine-tune

- [] Launch the full fine-tuning run on cloud GPU (1-3 hours)
- [] Monitor loss curves
- [] Save the resulting checkpoint
- [] Note cost and time for your records

Sunday 20 Jul (6 hrs) — Inference + RTC paper

- [] Load the fine-tuned SmolVLA checkpoint
- [] Run inference on a held-out episode
- [] Visualise predicted actions vs ground truth

- [] Read every line of the training script - understand each
- [] Write a 1-page report on what you did and observed
- [] Read RTC (Black et al. 2025); note async inference and action chunking

Week 5 — deliverable checklist and notes

- ☐ Fine-tuned SmolVLA checkpoint saved (cloud GPU)
- ☐ SmolVLA paper understood + 1-page summary
- ☐ RTC paper read
- ☐ Inference working: predicted vs ground-truth comparison

What I want to ask my supervisor

My notes

Week 6 — Perception, modular pipeline, and LLM planner

Week	6	Dates	21 - 27 July 2026
Target hours	~15 hours		
Theme	Open-vocab detection, depth-to-3D, scripted skills, LLM planner (closed-loop slice)		

Goal: by Sunday night you have the core of the modular pipeline working in simulation - an open-vocabulary detector returning boxes, a depth-to-3D conversion, scripted pick/place primitives, and an LLM planner that sequences them via tool calls - wired as a simple closed-loop slice. This is the architecture the project compares against the VLA, so this week is now central rather than peripheral.

Monday 21 Jul (1.5 hrs) — Open-vocabulary detection

- [] Install and run OWL-ViT (or GroundingDINO) on a sample image
- [] Pass a free-form text query; inspect returned boxes + confidence scores
- [] Understand the output: 2D pixel boxes only (no 3D, no class restriction)
- [] Threshold boxes by confidence; visualise on the image

Note: Reading link to lit review: contrast with closed-set YOLO (cannot see novel objects).

Tuesday 22 Jul (1.5 hrs) — Depth to 3D position

- [] Understand: detector gives a 2D pixel box; you need a 3D point
- [] Take the box centre pixel; read the depth value there (RGB-D)
- [] Convert pixel + depth to a 3D point in the camera frame (camera intrinsics)
- [] Write a small NumPy function: (pixel, depth) -> camera-frame XYZ

Wednesday 23 Jul (1.5 hrs) — Hand-eye calibration concept

- [] Read about fixed-camera (eye-to-hand) calibration
- [] Understand: the static transform from camera frame to robot base frame
- [] Chain it: object-in-camera -> object-in-base (one matrix multiply)
- [] Write the transform math in NumPy (4x4 homogeneous transform)

Note: This reuses the frame fluency from Week 2. Same idea as TF on your tank project.

Thursday 24 Jul (1.5 hrs) — Scripted motion primitives

- [] Write pick(xyz): move above -> descend -> close gripper -> lift
- [] Write place(xyz): move above target -> descend -> open -> retract
- [] Build these on top of the URController from Week 3
- [] Test pick-and-place in URSim with a hardcoded object position

Friday 25 Jul (1.5 hrs) — Anthropic SDK + tool calling

- [] Get an Anthropic API key; pip install anthropic
- [] Write a hello-world Claude call
- [] Read the tool use / function calling docs
- [] Define pick/place/move as tools the model can call
- [] Note rate limits and costs

Saturday 26 Jul (3 hrs) — LLM planner over real primitives

- [] System prompt describing available skills and detected objects
- [] LLM returns an ordered plan of tool calls (structured output)
- [] Parse the plan and dispatch to the scripted primitives
- [] Test: 'put the red block in the bin' -> detect -> plan -> execute (URSim)

Sunday 27 Jul (6 hrs) — Closed-loop modular slice + write-up

- [] Wire the full modular slice in simulation:

- [] ... capture frame -> open-vocab detect -> depth-to-3D -> object positions
- [] ... LLM plans skill sequence over detected objects
- [] ... execute primitives via URController in URSim
- [] Add a simple closed-loop step: re-detect before each grasp and update the target
- [] Document what works and what breaks (detection-stage vs grasp-stage failures)
- [] Update the git README with the current state

Note: Open-loop first; add the re-detect step only once open-loop runs end-to-end.

Week 6 — deliverable checklist and notes

- ☐ Open-vocabulary detector returning boxes on test images
- ☐ Depth-to-3D + hand-eye transform implemented (NumPy)
- ☐ Scripted pick/place primitives working in URSim
- ☐ LLM planner sequencing primitives via tool calls
- ☐ Closed-loop modular slice running end-to-end in simulation

What I want to ask my supervisor

My notes

Week 6.5 — Refinement, consolidation, and sign-off

Week	6.5	Dates	28 July - 4 August 2026
Target hours	~15 hours		
Theme	Refine proposal, consolidate code and notes, supervisor sign-off		

Goal: by 4 August you have a refined proposal, supervisor sign-off on scope, a tidy and documented codebase, and a clear priority list for Week 1 of the FYP. Since the proposal is already drafted, this week is about polishing and consolidating rather than writing from scratch, and absorbing any slippage from Weeks 1-6.

Mon 28 - Tue 29 Jul (1.5 hrs each (3 hrs)) — Refine the proposal

- [] Re-read the drafted proposal end to end
- [] Check it against any NTU EEE required format / section list
- [] Add or update the architecture diagram to match what you built in Week 6
- [] Tighten the methodology with concrete details learned during prep

Wednesday 30 Jul (1.5 hrs) — Send for sign-off

- [] Send the proposal to the supervisor
- [] Prepare 3-5 specific questions for the discussion
- [] Confirm: VLA model choice, lab GPU access, any budget

Thursday 31 Jul (1.5 hrs) — Consolidate the codebase

- [] Tidy the git repo: clear structure, working examples, README
- [] Ensure URController, DemoRecorder, detector, planner each have a runnable demo
- [] Write a short 'how to run each piece' note

Fri 1 - Sat 2 Aug (~6 hrs) — Catch up + revise

- [] Absorb anything that slipped from Weeks 1-6
- [] Revise the proposal on any early supervisor feedback
- [] Re-read notes on the 5 core papers (SmolVLA, OpenVLA, RT-2, RTC, EgoMimic)

Sunday 3 - Monday 4 Aug (~4 hrs) — Final prep for FYP start

- [] Write a 1-page 'current state' summary for Week 1 of the FYP
- [] Draft a priority list for the first two weeks of the FYP
- [] Re-read the FYP requirements document
- [] Set a weekly review cadence with the supervisor

Pre-FYP completion — deliverable checklist and notes

- ☐ Proposal refined and sent for sign-off
- ☐ Codebase consolidated: URController, DemoRecorder, detector, planner all runnable
- ☐ Fine-tuned SmolVLA checkpoint (tutorial dataset)
- ☐ Closed-loop modular slice working in simulation
- ☐ Notes on at least 5 deeply-read papers
- ☐ 1-page current-state summary + Week 1 priority list

What I want to ask my supervisor

My notes

Appendix A — Resources

Learning platforms

UR Academy (e-Series Core Track, NOT PolyScope X) - academy.universal-robots.com

Modern Robotics by Kevin Lynch (free textbook) - hades.mech.northwestern.edu

LeRobot tutorials and docs - github.com/huggingface/lerobot

Documentation to download

URScript Programming Manual e-Series (UR support site)

ur_rtde documentation (sdurobotics.gitlab.io/ur_rtde)

OWL-ViT and GroundingDINO model cards (HuggingFace)

Core papers to deep-read (priority order)

1. RT-2 (Brohan et al. 2023) - foundational VLA, generalisation claim

2. OpenVLA (Kim et al. 2024) - open-source baseline

3. SmolVLA (Shukor et al. 2025) - primary model

4. RTC (Black et al. 2025) - real-time / async control

5. Open X-Embodiment (Padalkar et al. 2023) - dataset and action formats

6. RT-1 (Brohan et al. 2022) - skim only, predecessor

Background only (lit review, not hands-on): EgoMimic (Kareer 2025), UMI (Chi 2024)

Tools to install

Python 3.10/3.11, Docker (URSim done), ur_rtde, LeRobot, ManiSkill3, an open-vocab detector (OWL-ViT/GroundingDINO), Anthropic SDK, Git

Cloud services

HuggingFace (LeRobot), Anthropic API (planner), Modal Labs or Colab Pro (GPU training)

Open questions for the supervisor

Lab GPU/workstation access for training; project budget; final VLA model choice; whether an existing ROS 2 setup exists for the UR5e.

End of document. Total period: 16 June 2026 - 4 August 2026. Revised to reflect the modular-vs-VLA comparison, demonstrations collected on the UR5E, and a closed-loop modular pipeline.